

LAPORAN PRATIKUM PEKAN 5
STRUKTUR DATA

Disusun Oleh:

Rifqi Aditya

2511533002

Dosen Pengampu :

Dr. Wahyudi, S.T, M.T

Asisten Praktikum :

Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum mata Struktur Data tentang *Queue*.

Laporan ini disusun untuk memenuhi salah satu tugas praktikum sekaligus sebagai sarana pembelajaran dalam memahami dasar-dasar pemrograman menggunakan bahasa Java. Melalui praktikum ini, penulis dapat mempelajari penggunaan Queue.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca, khususnya dalam memperdalam pemahaman tentang pemrograman dasar dengan Java.

Padang, 7 Mei 2025

Penulis

Daftar Pustaka

BAB I PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Tujuan	4
1.3 Manfaat	4
BAB II PEMBAHASAN	5
2.1 Single Linked List	5
2.1.1 Kode Program	5
2.1.2 Uraian Kode Program	5
2.2 Single Linked List – Penambahan	6
2.2.1 Kode Program	6
2.2.2 Uraian Kode Program	7
2.2.3 Output	8
2.3 Single Linked List – Pencarian	8
2.3.1 Kode Program	8
2.3.2 Uraian Kode Program	8
2.3.3 Output	9
2.4 Single Linked List – Hapus	9
2.4.1 Kode Program	9
2.4.2 Uraian Kode Program	10
2.4.3 Output	11
BAB III TUGAS	12
3.1 Main Class	12
3.1.1 Kode Program	12
3.1.2 Uraian Kode Program	12
3.2 Driver Class	13
3.2.1 Kode Program	13
3.2.2 Uraian Kode Program	14
3.3 Output	16
BAB III KESIMPULAN	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengelolaan data yang efisien merupakan hal yang sangat krusial. Salah satu struktur data fundamental yang sering digunakan adalah Linked List. Berbeda dengan Array yang memiliki ukuran statis dan alokasi memori yang berurutan, Single Linked List menawarkan fleksibilitas karena elemen-elemen yang disebut node dialokasikan secara dinamis di dalam memori.

Setiap node dalam Single Linked List menyimpan data dan sebuah referensi atau pointer yang menunjuk ke node berikutnya. Hal ini memungkinkan operasi penambahan (insert) dan penghapusan data dilakukan secara lebih efisien tanpa harus menggeser elemen lainnya. Pemahaman mendalam mengenai mekanisme pointer dan manajemen memori dinamis melalui Single Linked List menjadi landasan penting bagi mahasiswa sebelum mempelajari struktur data yang lebih kompleks.

1.2 Tujuan

1. Memahami konsep dasar struktur data Single Linked List.
2. Mampu mengimplementasikan Single Linked List dalam pemrograman menggunakan pointer.
3. Melatih kemampuan logika pemrograman dalam mengelola alokasi memori secara dinamis.

1.3 Manfaat

1. Meningkatkan pemahaman tentang struktur data, khususnya manajemen pointer dan node.
2. Membantu mahasiswa dalam mengembangkan kemampuan pemrograman yang lebih fleksibel dan efisien.
3. Meningkatkan kemampuan analisis dan pemecahan masalah dalam manipulasi struktur data linier.

BAB II

PEMBAHASAN

2.1 Single Linked List

2.1.1 Kode Program

```
1 package Pekan5_2511533002;
2
3 public class NodeSLL_2511533002 {
4     // node bagian data
5     int data_3002;
6     // pointer ke node berikutnya
7     NodeSLL_2511533002 next_3002;
8     // kosntruktor menginisialisasi node dengan data
9     public NodeSLL_2511533002(int data)
10    {
11        this.data_3002 = data;
12        this.next_3002 = null;
13    }
14 }
15
```

2.1.2 Uraian Kode Program

1. **NodeSLL_2511533002 next_3002;**
Mendeklarasikan atribut bernama next_3002 dengan tipe data NodeSLL_2511533002. Ini adalah pointer atau referensi yang akan menyimpan alamat memori dari node berikutnya dalam daftar.
2. **this.data_3002 = data;**
Baris ini memasukkan nilai dari parameter data ke dalam atribut data_3002 milik objek tersebut. Kata this merujuk pada atribut kelas untuk membedakannya dengan parameter.
3. **this.next_3002 = null;**
Baris ini menginisialisasi atribut next_3002 dengan nilai null. Hal ini karena setiap node baru yang dibuat secara default belum terhubung ke node mana pun.

2.2 Single Linked List – Penambahan

2.2.1 Kode Program

```
1 package Pekan5_2511533002;
2
3 public class TambahSLL_2511533002 {
4     public static NodeSLL_2511533002 insertAtFront(NodeSLL_2511533002 head_3002, int value_3002) {
5         NodeSLL_2511533002 new_node = new NodeSLL_2511533002(value_3002);
6         new_node.next_3002 = head_3002;
7         return new_node;
8     }
9     // fungsi menambahkan node di akhir SLL
10    public static NodeSLL_2511533002 InsertAtEnd(NodeSLL_2511533002 head_3002, int value_3002) {
11        // buat sebuah node dengan sebuah nilai
12        NodeSLL_2511533002 newNode = new NodeSLL_2511533002(value_3002);
13        // jika list kosong maka node menjadi head
14        if (head_3002 == null) {
15            return newNode;
16        }
17        // simpan head ke variabel sementara
18        NodeSLL_2511533002 last = head_3002;
19        // telusuri ke node akhir
20        while (last.next_3002 != null) {
21            last = last.next_3002;
22        }
23        // ubah pointer
24        last.next_3002 = newNode;
25        return head_3002;
26    }
27    static NodeSLL_2511533002 GetNode(int data_3002) {
28        return new NodeSLL_2511533002(data_3002);
29    }
30    static NodeSLL_2511533002 InsertPos(NodeSLL_2511533002 headNode_3002, int position_3002, int value_3002) {
31        NodeSLL_2511533002 head_3002 = headNode_3002;
32        if (position_3002 < 1)
33            System.out.println("Invalid Position");
34        if (position_3002 == 1) {
35            NodeSLL_2511533002 new_node = new NodeSLL_2511533002(value_3002);
36            new_node.next_3002 = head_3002;
37            return new_node;

```

```
38        } else {
39            while (position_3002-- != 0) {
40                if (position_3002 == 1) {
41                    NodeSLL_2511533002 newNode = GetNode(value_3002);
42                    newNode.next_3002 = headNode_3002.next_3002;
43                    headNode_3002.next_3002 = newNode;
44                    break;
45                }
46                headNode_3002 = headNode_3002.next_3002;
47            }
48            if (position_3002 != 1)
49                System.out.print("Posisi di luar jangkauan");
50        }
51        return head_3002;
52    }
53    public static void printList_2511533002(NodeSLL_2511533002 head_3002) {
54        NodeSLL_2511533002 curr_3002 = head_3002;
55        while (curr_3002.next_3002 != null) {
56            System.out.print(curr_3002.data_3002+"->");
57            curr_3002 = curr_3002.next_3002;
58        }
59        if (curr_3002.next_3002 == null) {
60            System.out.print(curr_3002.data_3002);
61            System.out.println();
62        }
63    }
64    public static void main(String[] args) {
65        // buat LinkedList 2->3->5->6
66        NodeSLL_2511533002 head_3002 = new NodeSLL_2511533002(2);
67        head_3002.next_3002 = new NodeSLL_2511533002(3);
68        head_3002.next_3002.next_3002 = new NodeSLL_2511533002(5);
69        head_3002.next_3002.next_3002.next_3002 = new NodeSLL_2511533002(6);
70        // cetak list asli
71        System.out.print("Senarai berantai awal:");
72        printList_2511533002(head_3002);

```

```
73        // tambahkan node baru di belakang
74        System.out.print("Tambah 1 simpul di belakang: ");
75        int data2_3002 = 7;
76        head_3002 = InsertAtEnd(head_3002, data2_3002);
77        // cetak update list
78        printList_2511533002(head_3002);
79        System.out.print("Tambah 1 simpul ke data 4 : ");
80        int data3_3002 = 4;
81        int pos_3002 = 4;
82        head_3002 = InsertPos(head_3002, pos_3002, data3_3002);
83        // cetak update list
84        printList_2511533002(head_3002);
85    }
86 }
```

2.2.2 Uraian Kode Program

1. **public static NodeSLL_2511533002 insertAtFront(NodeSLL_2511533002 head_3002, int value_3002) {**
Fungsi ini digunakan untuk menyisipkan *node* baru di posisi paling awal (sebelum head).
2. **new_node.next_3002 = head_3002;**
Menghubungkan pointer next dari *node* baru agar menunjuk ke head yang lama. Sekarang node baru ini berada di urutan paling depan.
3. **if (head_3002 == null) { return newNode; }**
Mengecek kondisi apakah Linked List masih kosong. Jika ya (head adalah null), maka *node* baru tersebut akan langsung dikembalikan dan menjadi head.
4. **NodeSLL_2511533002 last = head_3002;**
Membuat variabel sementara bernama last yang diinisialisasi dengan head. Variabel ini digunakan untuk menelusuri Linked List tanpa mengubah head aslinya.
5. **while (last.next_3002 != null) { last = last.next_3002; }**
Melakukan perulangan. Selama pointer next dari *node* last belum null, pointer last akan terus digeser ke *node* berikutnya.
6. **static NodeSLL_2511533002 InsertPos(NodeSLL_2511533002 headNode_3002, int position_3002, int value_3002) {**
Mendeklarasikan metode InsertPos yang menerima head, posisi target, dan nilai datanya.
7. **NodeSLL_2511533002 head_3002 = headNode_3002;**
Menyalin headNode_3002 ke variabel head_3002 agar posisi awal tidak hilang saat melakukan operasi pengembalian nilai.
8. **if (position_3002 == 1) {**
 NodeSLL_2511533002 new_node = new
 NodeSLL_2511533002(value_3002);
 new_node.next_3002 = head_3002;
 return new_node;}
Jika posisi yang diminta adalah 1 atau paling depan, Node baru ditaruh di depan dan langsung dikembalikan sebagai head baru.
9. **else {**
 while (position_3002-- != 0) {
Jika posisinya > 1, lakukan perulangan. Variabel position_3002 akan terus dikurangi untuk menghitung seberapa jauh kita harus menelusuri list.
 if (position_3002 == 1) {
Di dalam perulangan, jika kita sudah mencapai satu langkah sebelum posisi target, buat node baru.
 NodeSLL_2511533002 head_3002 = new NodeSLL_2511533002(2);
 head_3002.next_3002 = new NodeSLL_2511533002(3);
 head_3002.next_3002.next_3002 = new NodeSLL_2511533002(5);
 head_3002.next_3002.next_3002.next_3002 = new NodeSLL_2511533002(6);
Membuat struktur Linked List secara manual dengan menyambungkan pointer satu per satu.
12. **int data2_3002 = 7;**
 head_3002 = InsertAtEnd(head_3002, data2_3002);

Memanggil metode InsertAtEnd untuk memasukkan angka 7 di bagian paling akhir list.

13. `int data3_3002 = 4;`

`int pos_3002 = 4;`

`head_3002 = InsertPos(head_3002,pos_3002,data3_3002);`

Memanggil metode InsertPos untuk menyisipkan angka 4 pada posisi ke-4.

2.2.3 Output

```
Senarai berantai awal:2-->3-->5-->6
Tambah 1 simpul di belakang: 2-->3-->5-->6-->7
Tambah 1 simpul ke data 4 : 2-->3-->5-->4-->6-->7
```

2.3 Single Linked List – Pencarian

2.3.1 Kode Program

```
1 package Pekan5_2511533002;
2
3 public class PencarianSLL_2511533002 {
4     static boolean searchKey_3002(NodeSLL_2511533002 head_3002, int key_3002) {
5         NodeSLL_2511533002 curr_3002 = head_3002;
6         while (curr_3002 != null) {
7             if (curr_3002.data_3002 == key_3002)
8                 return true;
9             curr_3002 = curr_3002.next_3002; }
10        return false; }
11    public static void traversal_3002(NodeSLL_2511533002 head_3002) {
12        // mulai dari head
13        NodeSLL_2511533002 curr_3002 = head_3002;
14        //telusuri sampai pointer null
15        while (curr_3002 != null) {
16            System.out.print(" " + curr_3002.data_3002);
17            curr_3002 = curr_3002.next_3002; }
18        System.out.println(); }
19    public static void main(String[] Args) {
20        NodeSLL_2511533002 head_3002 = new NodeSLL_2511533002(14);
21        head_3002.next_3002 = new NodeSLL_2511533002(21);
22        head_3002.next_3002.next_3002 = new NodeSLL_2511533002(13);
23        head_3002.next_3002.next_3002.next_3002 = new NodeSLL_2511533002(30);
24        head_3002.next_3002.next_3002.next_3002.next_3002 = new NodeSLL_2511533002(10);
25        System.out.print("Penelusuran SLL : ");
26        traversal_3002(head_3002);
27        // data yang akan di cari
28        int key_3002 = 30;
29        System.out.print("Cari data "+ key_3002+ " = ");
30        if (searchKey_3002(head_3002, key_3002))
31            System.out.print("Ketemu");
32        else
33            System.out.println("Tidak ada");
34    }
```

2.3.2 Uraian Kode Program

1. `static boolean searchKey_3002(NodeSLL_2511533002 head_3002, int key_3002)`
{
Mendeklarasikan metode searchKey_3002 dengan tipe kembalian boolean. Metode ini menerima parameter dan angka yang ingin dicari.
2. `NodeSLL_2511533002 curr_3002 = head_3002;`
Kita menggunakan variabel bantuan ini agar titik head aslinya tidak berubah-ubah.
3. `while (curr_3002 != null) {`
Memulai perulangan. Selama curr_3002 belum null (artinya kita belum mencapai akhir list), lakukan pengecekan di dalam blok.
4. `if (curr_3002.data_3002 == key_3002)`
`return true;`

Mengecek apakah data pada *node* yang sedang ditunjuk saat ini sama dengan angka yang dicari. Jika sama, langsung kembalikan nilai true.

5. `NodeSLL_2511533002 head_3002 = new NodeSLL_2511533002(14);`
`head_3002.next_3002 = new NodeSLL_2511533002(21);`
`head_3002.next_3002.next_3002 = new NodeSLL_2511533002(13);`
`head_3002.next_3002.next_3002.next_3002 = new NodeSLL_2511533002(30);`
`head_3002.next_3002.next_3002.next_3002.next_3002 = new`
`NodeSLL_2511533002(10);`
Bagian ini membuat struktur Single Linked List secara manual.
6. `int key_3002 = 30;`
`System.out.print("Cari data "+ key_3002+ " = ");`
Menentukan angka yang ingin dicari (dalam hal ini 30) dan mencetak teks awalan ke layar.

2.3.3 Output

```
Penelusuran SLL : 14 21 13 30 10
Cari data 30 = Ketemu
```

2.4 Single Linked List – Hapus

2.4.1 Kode Program

```
1 package Pekan5_2511533002;
2
3 public class HapusSLL_2511533002 {
4 // fungsi untuk menghapus head
5 public static NodeSLL_2511533002 deletehead_3002(NodeSLL_2511533002 head_3002) {
6     // jika sll kosong
7     if (head_3002 == null)
8         return null;
9     // pindahkan head ke node berikutnya
10    head_3002 = head_3002.next_3002;
11    //return ke head baru
12    return head_3002;}
13
14
15 // fungsi menghapus node terakhir sll
16 public static NodeSLL_2511533002 removeLastNode_3002(NodeSLL_2511533002 head_3002) {
17     //jika list kosong, return null
18     if (head_3002 == null) {
19         return null;
20     }
21     // jika list satu node, hapus node dan return null
22     if (head_3002.next_3002 == null) {
23         // temukan node terakhir ke d+ua
24         return null;
25     }
26
27     NodeSLL_2511533002 secondLast_3002= head_3002;
28     while (secondLast_3002.next_3002.next_3002 != null) {
29         secondLast_3002 = secondLast_3002.next_3002;
30     }
31     //hapus node terakhir
32     secondLast_3002.next_3002 = null;
33     return head_3002;}
}
```

```

35 // fungsi menghapus node di posisi tertentu
36 public static NodeSLL_2511533002 deleteNode_3002(NodeSLL_2511533002 head_3002, int position_3002) {
37     NodeSLL_2511533002 temp_3002 = head_3002;
38     NodeSLL_2511533002 prev = null;
39
40     // jika linked list null
41     if (temp_3002 == null) {
42         return head_3002;
43     }
44
45     // kasus 1: head dihapus
46     if (position_3002 == 1) {
47         head_3002 = temp_3002.next_3002;
48         return head_3002;
49     }
50
51     // kasus 2: menghapus node di tengah
52     // telusuri ke node yang dihapus
53     for (int i = 1; temp_3002 != null && i < position_3002; i++) {
54         prev = temp_3002;
55         temp_3002 = temp_3002.next_3002;
56     }
57
58     // jika ditemukan, hapus node
59     if (temp_3002 != null) {
60         prev.next_3002 = temp_3002.next_3002;
61     } else {
62         System.out.println("Data tidak ada");
63     }
64
65     return head_3002;
66 }
67
68 // fungsi mencetak SLL
69 public static void printList_3002(NodeSLL_2511533002 head_3002) {

```

```

70     NodeSLL_2511533002 curr_3002 = head_3002;
71     while (curr_3002 != null && curr_3002.next_3002 != null) {
72         System.out.print(curr_3002.data_3002 + " -> ");
73         curr_3002 = curr_3002.next_3002;
74     }
75
76     if (curr_3002 != null) {
77         System.out.print(curr_3002.data_3002);
78     }
79     System.out.println();
80 }
81
82 public static void main(String[] args) {
83     NodeSLL_2511533002 head_3002 = new NodeSLL_2511533002(1);
84     head_3002.next_3002 = new NodeSLL_2511533002(2);
85     head_3002.next_3002.next_3002 = new NodeSLL_2511533002(3);
86     head_3002.next_3002.next_3002.next_3002 = new NodeSLL_2511533002(4);
87     head_3002.next_3002.next_3002.next_3002.next_3002 = new NodeSLL_2511533002(5);
88
89     System.out.println("List awal :");
90     printList_3002(head_3002);
91
92     head_3002 = deletehead_3002(head_3002);
93     System.out.println("List setelah head di hapus: ");
94     printList_3002(head_3002);
95
96     int position_3002 = 2;
97     head_3002 = deleteNode_3002(head_3002, position_3002);
98     System.out.println("List setelah node 2 di hapus : ");
99     printList_3002(head_3002);
100 }
101 }

```

2.4.2 Uraian Kode Program

1. **if (head_3002 == null) { return null; }**
Jika list kosong, kembalikan null.
2. **if (head_3002.next_3002 == null) { return null; }**
Jika list hanya berisi tepat satu node, menghapus node tersebut berarti membuat list menjadi kosong. Maka kembalikan null.
3. **while (secondLast_3002.next_3002.next_3002 != null) {**
 secondLast_3002 = secondLast_3002.next_3002;
Melakukan perulangan. Kondisi next.next != null berarti perulangan akan berhenti tepat di satu node sebelum node terakhir.
4. **secondLast_3002.next_3002 = null;**

Memutus hubungan dengan node terakhir. Dengan menjadikan pointer next milik node kedua-dari-belakang menjadi null, maka node yang paling ujung resmi terhapus dari rantai SLL.

5. **NodeSLL_2511533002 temp_3002 = head_3002;**

NodeSLL_2511533002 prev = null;

Pada temp_3002 digunakan untuk menunjuk node yang akan dihapus. Lalu prev digunakan untuk menunjuk node sebelum temp_3002. Ini penting untuk menghubungkan kembali putusan list nanti.

6. **if (position_3002 == 1) {**

head_3002 = temp_3002.next_3002;

return head_3002; }

Jika posisi yang diminta adalah 1 (paling depan), maka logikanya sama persis dengan metode deletehead. Geser head ke node kedua, lalu kembalikan.

7. **for (int i = 1; temp_3002 != null && i < position_3002; i++) {**

prev = temp_3002;

temp_3002 = temp_3002.next_3002;}

Jika posisinya di tengah/akhir, gunakan loop for. Loop ini bergeser mencari target. Di setiap pergeseran, pointer prev akan menyimpan posisi temp lama, dan temp bergeser ke next. Saat loop selesai, temp akan berada di node yang akan dihapus, dan prev persis di belakangnya.

8. **head_3002 = deletehead_3002(head_3002);**

Memanggil fungsi penghapus head. Angka 1 terhapus.

9. **int position_3002 = 2;**

head_3002 = deleteNode_3002(head_3002, position_3002);

Meminta untuk menghapus node pada posisi ke-2.

2.4.3 Output

```
List awal :  
1 -> 2 -> 3 -> 4 -> 5  
List setelah head di hapus:  
2 -> 3 -> 4 -> 5  
List setelah node 2 di hapus :  
2 -> 4 -> 5
```

BAB III

TUGAS

3.1 Main Class

3.1.1 Kode Program

```
1 package Pekan5_2511533002;
2 public class Kendaraan_2511533002 {
3     private String namaPemilik_3002;
4     private String jenisKendaraan_3002;
5     private int nomorAntrean_3002;
6     Kendaraan_2511533002 next_3002;
7
8     public Kendaraan_2511533002(String nama_3002, String jenis_3002, int antrean_3002) {
9         this.namaPemilik_3002 = nama_3002;
10        this.jenisKendaraan_3002 = jenis_3002;
11        this.nomorAntrean_3002 = antrean_3002;
12        this.next_3002 = null;
13    }
14
15    public String getNamaPemilik_3002() { return namaPemilik_3002; }
16    public String getJenisKendaraan_3002() { return jenisKendaraan_3002; }
17    public int getNomorAntrean_3002() { return nomorAntrean_3002; }
18    public Kendaraan_2511533002 getNext_3002() { return next_3002; }
19
20    public void setNamaPemilik_3002(String nama_3002) { this.namaPemilik_3002 = nama_3002; }
21    public void setJenisKendaraan_3002(String jenis_3002) { this.jenisKendaraan_3002 = jenis_3002; }
22    public void setNomorAntrean_3002(int antrean_3002) { this.nomorAntrean_3002 = antrean_3002; }
23    public void setNext_3002(Kendaraan_2511533002 next_3002) { this.next_3002 = next_3002; }
24 }
```

3.1.2 Uraian Kode Program

1. Konstruktor

- `public Kendaraan_2511533002(String nama_3002, String jenis_3002, int antrean_3002) {`
 `this.namaPemilik_3002 = nama_3002;`
 `this.jenisKendaraan_3002 = jenis_3002;`
 `this.nomorAntrean_3002 = antrean_3002;`
 `this.next_3002 = null;`

fungsi khusus yang otomatis dijalankan pertama kali saat kamu membuat objek kendaraan baru

2. Method Selektor

- `public String getNamaPemilik_3002() { return namaPemilik_3002; }`
 `public String getJenisKendaraan_3002() { return jenisKendaraan_3002; }`
 `public int getNomorAntrean_3002() { return nomorAntrean_3002; }`
 `public Kendaraan_2511533002 getNext_3002() { return next_3002; }`
 - mengembalikan nilai dari masing-masing atribut ketika dipanggil.

3. Method Mutator

- `public void setNamaPemilik_3002(String nama_3002) { this.namaPemilik_3002 = nama_3002; }`
 `public void setJenisKendaraan_3002(String jenis_3002) {`
 `this.jenisKendaraan_3002 = jenis_3002; }`
 `public void setNomorAntrean_3002(int antrean_3002) {`
 `this.nomorAntrean_3002 = antrean_3002; }`

```

        public void setNext_3002(Kendaraan_2511533002 next_3002) { this.next_3002
= next_3002; }
    }

```

digunakan jika kita perlu *mengubah* atau memperbarui nilai dari atribut yang sudah ada secara aman.

3.2 Driver Class

3.2.1 Kode Program

```

1 package Pekan5_2511533002;
2 import java.util.Scanner;
3
4 public class KendaraanDriver_2511533002 {
5     private Kendaraan_2511533002 head_3002 = null;
6     private int counter_3002 = 0;
7
8     public void daftarkanKendaraan_3002(String nama_3002, String jenis_3002) {
9         counter_3002++;
10        Kendaraan_2511533002 nodeBaru_3002 = new Kendaraan_2511533002(nama_3002, jenis_3002, counter_3002);
11
12        if (head_3002 == null) {
13            head_3002 = nodeBaru_3002;
14        } else {
15            Kendaraan_2511533002 temp_3002 = head_3002;
16            while (temp_3002.getNext_3002() != null) {
17                temp_3002 = temp_3002.getNext_3002();
18            }
19            temp_3002.setNext_3002(nodeBaru_3002);
20        }
21        System.out.println("Kendaraan berhasil masuk antrean! Nomor Antrean: " + counter_3002);
22    }
23
24    public void layaniKendaraan_3002() {
25        if (head_3002 == null) {
26            System.out.println("Antrean kosong, tidak ada kendaraan untuk dilayani.");
27        } else {
28            System.out.println("Melayani kendaraan: " + head_3002.getNamaPemilik_3002() + " (" + head_3002.getJenisKendaraan_3002() + "
29            head_3002 = head_3002.getNext_3002();
30        }
31    }
32
33    public void tampilkanAntrean_3002() {
34        if (head_3002 == null) {
35            System.out.println("Antrean saat ini kosong.");
36        } else {
37            System.out.println("\n--- Daftar Antrean Car Wash ---");
38            Kendaraan_2511533002 temp_3002 = head_3002;
39            int posisi_3002 = 1;
40            while (temp_3002 != null) {
41                System.out.println(posisi_3002 + ". [" + temp_3002.getNomorAntrean_3002() + "] " +
42                temp_3002.getNamaPemilik_3002() + " - " + temp_3002.getJenisKendaraan_3002());
43                temp_3002 = temp_3002.getNext_3002();
44                posisi_3002++;
45            }
46        }
47    }
48
49    public void cariKendaraan_3002(String namaCari_3002) {
50        if (head_3002 == null) {
51            System.out.println("Antrean kosong.");
52            return;
53        }
54        Kendaraan_2511533002 temp_3002 = head_3002;
55        boolean ditemukan_3002 = false;
56        while (temp_3002 != null) {
57            if (temp_3002.getNamaPemilik_3002().equalsIgnoreCase(namaCari_3002)) {
58                System.out.println("Data Ditemukan: Nomor " + temp_3002.getNomorAntrean_3002() +
59                ", Pemilik: " + temp_3002.getNamaPemilik_3002() +
60                ", Jenis: " + temp_3002.getJenisKendaraan_3002());
61                ditemukan_3002 = true;
62            }
63            temp_3002 = temp_3002.getNext_3002();
64        }
65        if (!ditemukan_3002) System.out.println("Data dengan nama '" + namaCari_3002 + "' tidak ditemukan.");
66    }
67
68    public void cekStatus_3002() {
69        if (head_3002 == null) {
70            System.out.println("Antrean kosong.");
71        } else {
72            int total_3002 = 0;
73            Kendaraan_2511533002 temp_3002 = head_3002;

```

```

74     while (temp_3002 != null) {
75         total_3002++;
76         temp_3002 = temp_3002.getNext_3002();
77     }
78     System.out.println("Total kendaraan dalam antrean: " + total_3002);
79     System.out.println("Kendaraan paling depan: " + head_3002.getNamaPemilik_3002());
80 }
81 }
82
83 public static void main(String[] args) {
84     KendaraanDriver_2511533002 program_3002 = new KendaraanDriver_2511533002();
85     Scanner input_3002 = new Scanner(System.in);
86     int pilihan_3002;
87
88     do {
89         System.out.println("\n=== Antrean Car Wash NIM: 2511533002 ===");
90         System.out.println("1. Tambah Antrean Kendaraan");
91         System.out.println("2. Layani Kendaraan (Selesai Cuci)");
92         System.out.println("3. Lihat Seluruh Antrean");
93         System.out.println("4. Cari Data Pemilik");
94         System.out.println("5. Cek Status Antrean");
95         System.out.println("6. Keluar");
96         System.out.print("Pilihan: ");
97         pilihan_3002 = input_3002.nextInt();
98         input_3002.nextLine();
99
100        switch (pilihan_3002) {
101            case 1:
102                System.out.print("Masukkan Nama Pemilik: ");
103                String n_3002 = input_3002.nextLine();
104                System.out.print("Masukkan Jenis Kendaraan (Mobil/Motor): ");
105                String j_3002 = input_3002.nextLine();
106                program_3002.daftarkanKendaraan_3002(n_3002, j_3002);
107                break;
108            case 2:
109                program_3002.layaniKendaraan_3002();

```

```

110            break;
111            case 3:
112                program_3002.tampilkanAntrean_3002();
113                break;
114            case 4:
115                System.out.print("Cari Nama Pemilik: ");
116                String c_3002 = input_3002.nextLine();
117                program_3002.cariKendaraan_3002(c_3002);
118                break;
119            case 5:
120                program_3002.cekStatus_3002();
121                break;
122            case 6:
123                System.out.println("Program selesai.");
124                break;
125            default:
126                System.out.println("Pilihan tidak valid!");
127        }
128    } while (pilihan_3002 != 6);
129    input_3002.close();
130 }
131 }

```

3.2.2 Uraian Kode Program

1. } else {

```

    Kendaraan_2511533002 temp_3002 = head_3002;
    while (temp_3002.getNext_3002() != null) {
        temp_3002 = temp_3002.getNext_3002();
    }
    temp_3002.setNext_3002(nodeBaru_3002);

```

Jika antrean sudah ada isinya, kita buat pointer temp_3002 untuk menelusuri dari *head* sampai ke ujung belakang antrean (saat getNext_3002() == null). Setelah sampai di ujung, kita sambungkan kendaraan terakhir tersebut ke nodeBaru_3002 menggunakan metode setNext_3002().

2. } else {

```

    System.out.println("Melayani kendaraan: " +
    head_3002.getNamaPemilik_3002() + " (" + head_3002.getJenisKendaraan_3002() +
    ")");
    head_3002 = head_3002.getNext_3002();

```

Jika ada isinya, cetak nama dan jenis kendaraan dari *node* yang berada di head. Pindahkan head_3002 ke kendaraan di belakangnya (getNext_3002()). Ini secara otomatis menghapus kendaraan pertama dari daftar antrean.

3. **Kendaraan_2511533002 temp_3002 = head_3002;**
int posisi_3002 = 1;
while (temp_3002 != null) {
System.out.println(posisi_3002 + ". [" + temp_3002.getNomorAntrean_3002() + "]" +
temp_3002.getNamaPemilik_3002() + " - " + temp_3002.getJenisKendaraan_3002());
temp_3002 = temp_3002.getNext_3002(); posisi_3002++;

Jika ada data, gunakan variabel temp_3002 untuk menelusuri list dari head sampai null. Variabel posisi_3002 digunakan sekadar untuk mencetak urutan tampilan (1, 2, 3, dst). Setiap kali iterasi jalan, data dicetak dan pointer temp_3002 digeser ke *node* selanjutnya.

4. **if (!ditemukan_3002) System.out.println("Data dengan nama " + namaCari_3002 + "**
tidak ditemukan.");

Jika setelah ditelusuri bendera masih bernilai false, berarti nama tidak ada di dalam antrean.

5. **if (head_3002 == null) {**
System.out.println("Antrean kosong.");
} else {
int total_3002 = 0;
Kendaraan_2511533002 temp_3002 = head_3002;
while (temp_3002 != null) {
total_3002++;
temp_3002 = temp_3002.getNext_3002();}
System.out.println("Total kendaraan dalam antrean: " + total_3002);
System.out.println("Kendaraan paling depan: " +
head_3002.getNamaPemilik_3002());

Jika ada antrean, program akan menghitung jumlah node dengan melakukan perulangan while dan terus menambahkan total_3002 sampai mencapai ujung list. Kemudian menampilkan total jumlah kendaraan dan memanggil nama kendaraan di posisi head.

3.3 Output

```
=== Car Wash Premium ===
1. Tambah Antrean Kendaraan
2. Layani Kendaraan (Selesai Cuci)
3. Lihat Seluruh Antrean
4. Cari Data Pemilik
5. Cek Status Antrean
6. Keluar
Pilihan: 1
Masukkan Nama Pemilik: budi
Masukkan Jenis Kendaraan (Mobil/Motor): mobil
Kendaraan berhasil masuk antrean! Nomor Antrean: 1
```

```
=== Car Wash Premium ===
1. Tambah Antrean Kendaraan
2. Layani Kendaraan (Selesai Cuci)
3. Lihat Seluruh Antrean
4. Cari Data Pemilik
5. Cek Status Antrean
6. Keluar
Pilihan: 1
Masukkan Nama Pemilik: tayo
Masukkan Jenis Kendaraan (Mobil/Motor): motor
Kendaraan berhasil masuk antrean! Nomor Antrean: 2
```

```
=== Car Wash Premium ===
1. Tambah Antrean Kendaraan
2. Layani Kendaraan (Selesai Cuci)
3. Lihat Seluruh Antrean
4. Cari Data Pemilik
5. Cek Status Antrean
6. Keluar
Pilihan:
```

```
=== Car Wash Premium ===
1. Tambah Antrean Kendaraan
2. Layani Kendaraan (Selesai Cuci)
3. Lihat Seluruh Antrean
4. Cari Data Pemilik
5. Cek Status Antrean
6. Keluar
Pilihan: 3
```

```
--- Daftar Antrean Car Wash ---
1. [1] budi - mobil
2. [2] tayo - motor
```

```
=== Car Wash Premium ===
1. Tambah Antrean Kendaraan
2. Layani Kendaraan (Selesai Cuci)
3. Lihat Seluruh Antrean
4. Cari Data Pemilik
5. Cek Status Antrean
6. Keluar
Pilihan: 4
Cari Nama Pemilik: tayo
Data Ditemukan: Nomor 2, Pemilik: tayo, Jenis: motor
```



```
=== Car Wash Premium ===
1. Tambah Antrean Kendaraan
2. Layani Kendaraan (Selesai Cuci)
3. Lihat Seluruh Antrean
4. Cari Data Pemilik
5. Cek Status Antrean
6. Keluar
Pilihan: 2
Melayani kendaraan: budi (mobil)
```

```
=== Car Wash Premium ===
1. Tambah Antrean Kendaraan
2. Layani Kendaraan (Selesai Cuci)
3. Lihat Seluruh Antrean
4. Cari Data Pemilik
5. Cek Status Antrean
6. Keluar
Pilihan: 5
Total kendaraan dalam antrean: 1
Kendaraan paling depan: tayo
```

```
=== Car Wash Premium ===
1. Tambah Antrean Kendaraan
2. Layani Kendaraan (Selesai Cuci)
3. Lihat Seluruh Antrean
4. Cari Data Pemilik
5. Cek Status Antrean
6. Keluar
Pilihan: 6
Program selesai.
```

BAB III

KESIMPULAN

Single Linked List merupakan struktur data dinamis yang menawarkan fleksibilitas tinggi dalam alokasi memori jika dibandingkan dengan struktur data statis seperti array. Dalam operasi penambahan data, SLL terbukti sangat efisien karena prosesnya tidak memerlukan pergeseran elemen-elemen lain. Penambahan data baik itu di posisi awal, di posisi akhir, maupun disisipkan pada indeks tertentu hanya melibatkan pembuatan node baru dan manipulasi arah pointer untuk merangkaikan node tersebut ke dalam list. Namun, fleksibilitas bentuk ini memberikan kompromi pada operasi pencarian data. Karena SLL tidak memiliki kemampuan akses acak, pencarian data harus selalu dilakukan secara linier dengan menelusuri node satu per satu dimulai dari head, yang berarti waktu pemrosesan akan semakin lama seiring dengan bertambahnya jumlah data di dalam list.

Sementara itu, operasi penghapusan data pada Single Linked List juga sangat bergantung pada ketepatan manajemen pointer. Untuk menghapus suatu data dari list, program harus menemukan node target sekaligus melacak node yang berada tepat sebelumnya. Dengan mengubah pointer dari node sebelumnya agar langsung node target dan menunjuk ke node setelahnya, node target akan terlepas dari rantai SLL. Secara keseluruhan, pemahaman yang mendalam mengenai mekanisme penambahan, pencarian, dan penghapusan ini membuktikan bahwa Single Linked List adalah struktur data yang sangat ideal untuk program yang terus berubah ukurannya secara dinamis, di mana operasi modifikasi data lebih diutamakan daripada kecepatan akses langsung ke data tertentu.